Kairos: Schedule Exploration for Multicore RTL Simulation

Anonymous Author(s)

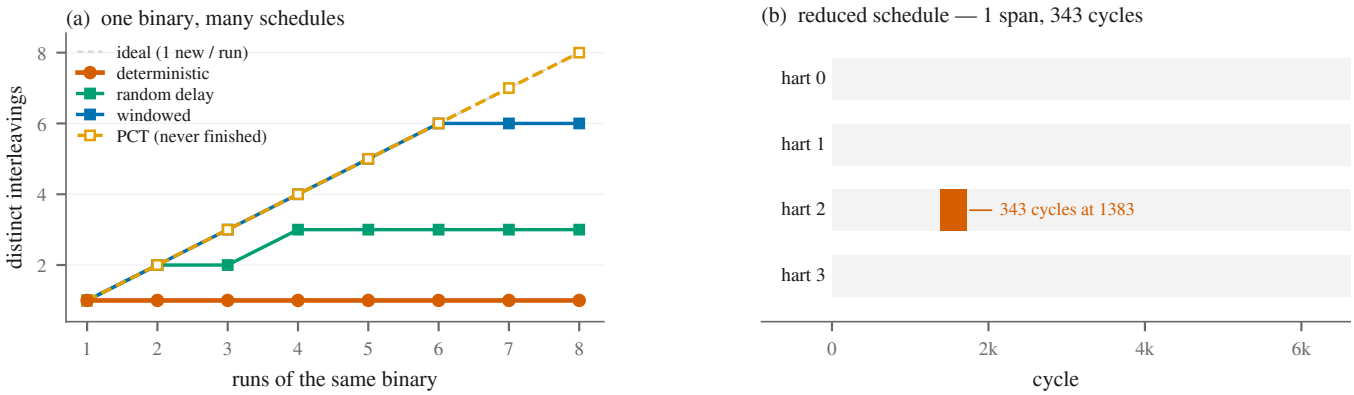

Figure 1: The claim, on one workload (`mt-spinwait`). (a) Eight runs of a deterministic RTL simulation explore one interleaving; the same eight runs under schedule exploration explore up to eight. (b) The failure that windowed delay found, after automatic reduction: hart 2 held for 343 cycles at cycle 1383. Replaying that one-line file reproduces the livelock; after the race was fixed, the same file completes cleanly.

Abstract

Cycle-accurate RTL simulation of a multicore is deterministic: a test program explores exactly one interleaving, no matter how many times it is run. Silicon does not have this limitation—timing jitter is why post-silicon multicore validation works at all. We measure the resulting coverage wall: reverting a real dual-Unique coherence bug changes nothing observable on five directed reproducers, because the schedule never varies. *Kairos* restores schedule diversity pre-silicon by delaying harts under a reproducible policy. The RTL cost is one bit per core gating an existing handshake; compiled out, the generated netlist is identical to the original design (0 differing lines). Because a stall is always architecturally legal, every induced schedule is one the hardware could reach unaided—there is no false-positive class from the mechanism. On a quad-core out-of-order RISC-V, eight runs of a conventional regression explore one interleaving; the same eight runs under Kairos explore up to eight. Two latent races in the shared SMP boot code—missed by years of green CI—were found in under ten runs of workloads below 7,000 cycles and reduced to one named-hart delay. Both are software races, not RTL bugs.

CCS Concepts

• **Hardware → Hardware validation**; *Simulation and emulation.*

Keywords

RTL simulation, multicore, schedule exploration, cache coherence

DAC '27, San Jose, CA, USA

ACM ISBN 978-1-4503-XXXX-X/27/07
https://doi.org/XXXXXXX.XXXXXXX

ACM Reference Format:
Anonymous Author(s). 2027. Kairos: Schedule Exploration for Multicore RTL Simulation. In *Proceedings of 64th ACM/IEEE Design Automation Conference (DAC '27).* ACM, New York, NY, USA, 7 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

A cycle-accurate RTL simulator is a pure function of its inputs. Given the same binary, the same reset, and the same device tree, a multicore simulation produces the same interleaving of memory events on every run. Repeating a regression a thousand times therefore explores one schedule, not a thousand.

Silicon does not have this problem. Clock jitter, interrupt arrival, DRAM refresh, voltage droop and thermal throttling re-order the machine on every execution, which is why post-silicon multicore validation keeps finding coherence and memory-ordering bugs that pre-silicon simulation ran past [5, 10]. That variation is a *coverage source.* Simulation throws it away precisely at the stage where bugs are cheapest to fix.

The gap is not hypothetical. Five directed reproducers for a dual-Unique coherence violation in the processor studied here—two L1s holding the same line Unique at once—passed on both the buggy and the fixed RTL. The only known activation was an 871-million-cycle Linux boot. Measured directly: with the fix reverted, all five workloads produced *bit-identical* cache tag-transition counts to the fixed model (Section 3). Removing a real coherence bug changed nothing observable, because the schedule never varied.

This paper treats that determinism as a coverage wall and asks whether it can be lifted without putting a Linux-booting design at risk. *Kairos* delays harts according to a controlled, reproducible policy so that one test program samples many interleavings. The RTL change is one input per core, gating a handshake the frontend already de-asserts on every I-cache miss (Figure 3). Tied low, or compiled out, the design is the original design—measured, not asserted. Because a core is always permitted to stall, every schedule

Kairos induces is a schedule the unmodified hardware could reach, and any failure it finds is a real failure.

The ideas are imported from software concurrency testing [2, 12, 13] and the paper is generous about that debt. What does not transfer—no blocking signal, a scheduling-point count in the hundreds of thousands, a livelock oracle that must discount the tool's own stalls—is measured, and is the interesting part of the work.

This paper makes four contributions:

- **The determinism wall, measured.** Eight runs of a conventional RTL regression explore one interleaving, on every workload we ran (Figure 1a, Figure 4). Reverting a real coherence bug does not change a single tag transition (Section 3).
- **A sound perturbation.** One bit per core, four lines of logic, netlist-identical when compiled out. Stalling cannot manufacture a state the design could not reach unaided.
- **A four-policy ablation on one DUT.** Deterministic regression, UVM-style random delay, PCT, and windowed delay: 96 runs, two coverage metrics, time to first finding (Section 5).
- **Two reduced, replayable findings** in boot code that every SMP test in the repository links. Both survived years of a green deterministic regression. Both are software races, not RTL bugs; stating that is part of the result.

2 Related Work

Figure 2 is the map. Almost all dynamic hardware verification varies the *program*. Kairos varies the *schedule* of a fixed program. Those multiply; they do not compete. Table 1 records stimulus, coverage, oracle, and the baseline each paper actually ran on one DUT—the comparison those papers make, not a cross-DUT bug count. The ablation we run here is in Section 5.2.

Program fuzzers for RTL.. RFUZZ [9] mutates mux inputs under mux-toggle coverage. DifuzzRTL [6] guides on control-register coverage and found 16 new bugs in Mor1kx, Rocket and BOOM. TheHuzz [8] generates assembly against a golden model: 11 bugs, 8 of them new, on four cores, 3.33× the speed of DifuzzRTL. ProcessorFuzz [3] uses CSR transitions: 8 new bugs plus one in a reference model, 1.23× vs DifuzzRTL. Cascade [14] generates long valid RISC-V programs and detects bugs by non-termination: 37 new bugs in five CPUs, and 28.2–97× the coverage of prior fuzzers. MorFuzz [16] reports 19 bugs. All of these evaluations are single-core. None controls the interleaving of a coherent multicore, and none checks live cache state or stall-aware liveness.

Multicore oracles and MCM testers. DiffTest [17] and DiffTest-H [11] are commit-level lockstep, including multicore; DiffTest-H reports 151 complex bugs in XiangShan. That is oracle layer 1 here, and we keep it. It is silent on hangs and on illegal microarchitectural states that have not yet corrupted a commit—the dual-Unique bug was a Linux hang ~10^8 cycles later. TSOtool [5] and McVerSi [4] vary the *program* and check values against a memory model, on silicon or gem5, where timing already varies. McVerSi found 2 new gem5 bugs and all 11 studied bugs in one day, against 2/11 for litmus tests. MTraceCheck [10] is post-silicon. Litmus tests [1] enumerate allowed outcomes. Kairos is the complement: fix the program, vary the schedule, observe the microarchitecture, minimise the result.

Software schedule exploration. CHESS [12, 13] and PCT [2] explore thread schedules; delta debugging [18] minimises failing inputs. The ideas are theirs. Section 4 states what breaks in hardware. UVM testbenches randomise interface delays [7] with no coverage notion; Kairos controls the cores.

3 The Determinism Wall

The strongest single argument in this paper was produced as a by-product of a failed experiment. A continuous SWMR probe over all four L1 tag arrays was pointed at five SMP workloads, once on the fixed RTL and once with the dual-Unique fix reverted. Tag-state transition counts over 6 million cycles per workload were *identical to the digit*: 4,091 / 4,158 / 26,090 / 160,148 / 139,716 on `mt-fencei`, `mt-crosscall`, `mt-llist`, `mt-seqlock`, `mt-lrsc`. The programs were not wrong; each ran one schedule, and that schedule did not contain the race. The Linux boot found the bug by brute force at ~8.7×10^8 cycles—coverage by accident.

The same wall appears as a coverage number. Under the deterministic policy, eight runs of each of three SMP micros produce one distinct interleaving (Figure 4, top row of each panel). Novelty rate is 1/8. That is what a conventional RTL regression does today, written as a fraction.

4 Kairos

Kairos is a host-side scheduler around a one-bit RTL hook (Figure 3). Everything above the DUT boundary—policies, coverage, oracles, reduction, replay—is written against an eleven-method interface (`dut.h`). Porting to another multicore RTL model means implementing that interface and adding one stall bit per hart.

4.1 Perturbation and soundness

The RTL change is one input per core, `scheduleStall`, ANDed into the fetch→decode handshake (Figure 3a). Holding it high is behaviourally identical to fetch not being ready, a condition the design already produces on every I-cache miss. So:

- A stall can only *delay* a hart. It cannot skip work, force progress, win an arbitration, or change a value.
- Every induced schedule is one the unmodified design could reach on its own. A failure under Kairos is a failure of the design (or of the software running on it), not an artefact of the tool. There is no false-positive class from the mechanism.
- With the mask held at zero the design behaves as it does without Kairos. With the feature compiled out, the generated Verilog is identical to a build that never heard of Kairos.

That last claim took four tries: Chisel derives identifiers from elaboration order, so a refactor that looks equivalent is not netlist-neutral. Writing the disabled path as the literal original statement is the only form that gives 0 differing lines. This separates the mechanism from fault injection: a stall produces only states the silicon could reach.

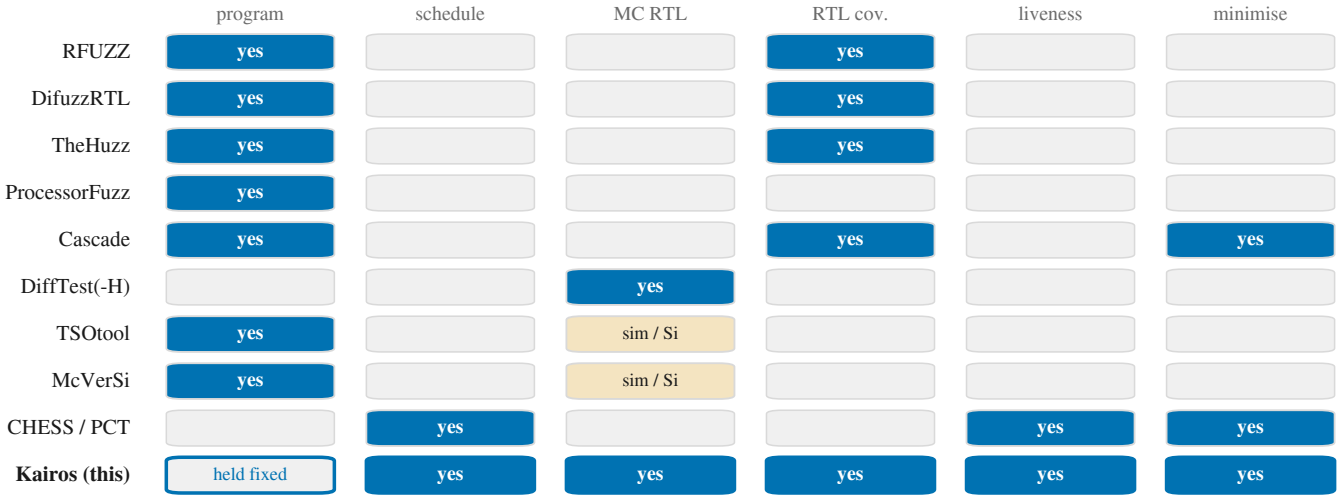

Figure 2: What each tool does. Blue is yes; tan is silicon or full-system simulation, not RTL. "RTL cov." is mux, register, VCS, or live cache coverage. Cascade minimises *programs*; CHESS/PCT and Kairos minimise *schedules*. No row except Kairos is yes in schedule + multicore RTL + RTL coverage together.

Table 1: How each paper evaluates: stimulus, coverage, oracle, and the baseline it actually ran on the same DUT. Bug counts belong in each paper's own findings table, not here.

Tool	Stimulus	Coverage metric	Oracle	Same-DUT baseline
RFUZZ [9]	mux bit stream	mux-toggle	assertions	vs. random
DifuzzRTL [6]	assembly	control-register	Spike / or1ksim	vs. RFUZZ (A_{12})
TheHuzz [8]	assembly	stmt/branch/toggle (VCS)	Spike / or1ksim	vs. random + DifuzzRTL, n=10
ProcessorFuzz [3]	assembly	CSR transitions	Spike / Dromajo	TTE vs. DifuzzRTL, 48 h
Cascade [14]	long RV programs	mux + register + simulator	non-termination	RFUZZ reimpl. + DifuzzRTL docker
McVerSi [4]	GP programs	protocol transitions	MCM checker	vs. random + litmus
Kairos	schedule of a fixed binary	distinct schedules, order pairs	result + liveness + SWMR	vs. det. + random + PCT

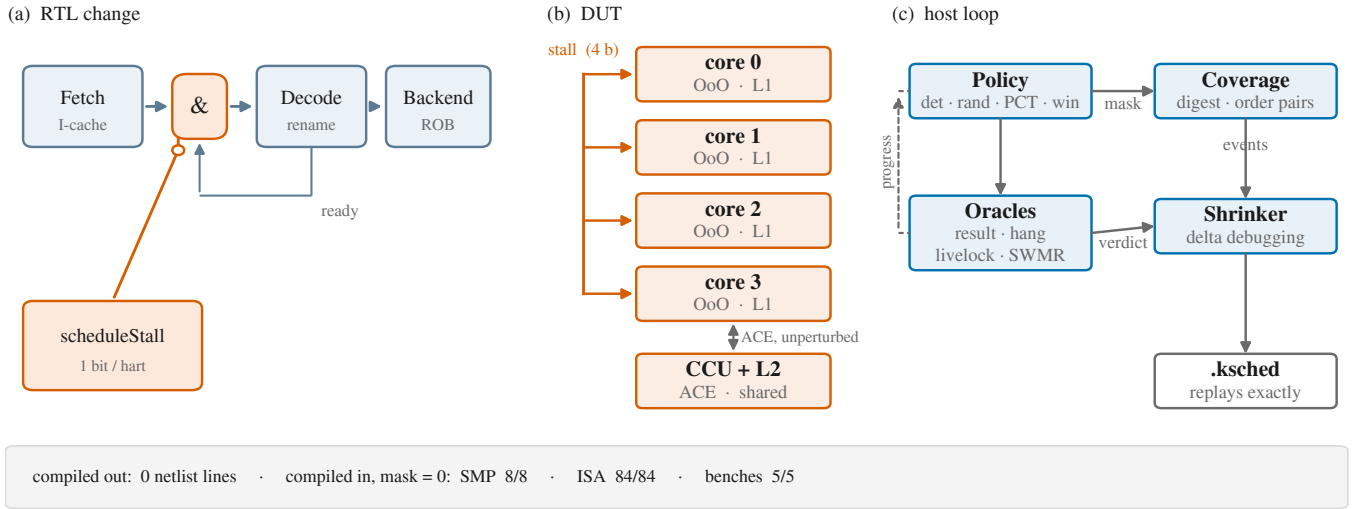

Figure 3: The mechanism. (a) One AND into a handshake the frontend already de-asserts; a stall is fetch-not-ready. (b) The stall bit is a bus into every hart; observation is read-only. (c) The host loop. Footer: gates that protect the Linux-booting design, measured.

4.2 Policies

A policy decides, each cycle, which harts to hold. Every policy is a pure function of (*seed*, *cycle*, *progress*)—no wall-clock, no I/O—so a finding is reproducible from its seed alone. Four policies ship, and three of them exist to be argued with:

- `deterministic` never stalls. It is the control: what a conventional regression does today.
- `random`(p) holds each hart independently with probability p per cycle. This is the honest strawman—roughly what

a UVM testbench does with randomised delays [7]. If it matches PCT, PCT is not earning its complexity.

- `pct`(d) is PCT [2] transferred to hardware: random hart priorities plus $d-1$ random priority-change points, with bounded leadership tenure (below).
- `windowed`(w) holds one hart for one contiguous window. Wide races (a walker writeback outliving a peer's refill) need a contiguous delay; per-cycle coin flips reopen and reclose them immediately.

What does not transfer from PCT.. Software PCT runs the highest-priority enabled thread; the runtime knows when a thread blocks. Hardware has no such signal, and "has not retired" is not a substitute: a hart spinning on a flag retires forever and would hold the machine forever. Measured, before bounded tenure: `pct:3` left two harts with zero retired instructions after 662,901 cycles. Leadership therefore expires after 4,096 cycles. This adds priority changes the original guarantee does not account for, and is stated rather than hidden.

Three further caveats, all load-bearing. (1) In software, k is the number of scheduling steps; in hardware every cycle is one, so k is far larger and the bound correspondingly weaker. (2) The depth of a coherence race is not obviously the same notion as the depth of a data race. (3) Kairos transfers PCT's *structure* and measures how the bound behaves; it does not inherit the guarantee.

Calibration, not assumption. A policy has to know *when* to perturb. Spreading delays over the cycle *budget* is wrong whenever the budget is a safety cap rather than a prediction. The first working build placed one delay window uniformly over a 400,000-cycle budget for a 6,629-cycle workload, so in ~98% of runs the window landed after the program had already ended. Six runs, six identical digests, a campaign that looked perfectly healthy while exploring nothing. Kairos now measures the unperturbed run length first and spreads over that. The same run is the control: if the workload does not pass unperturbed, the campaign refuses to start.

4.3 Oracles

Four layers run on every cycle of every run. Each answers a different question, and each is reported separately.

(1) **Result.** Did the workload compute the right answer?

(2) **Liveness.** Hang: a hart stops retiring. Livelock: every hart retires and the workload never finishes. Both produce no values, so layer 1 and every value-based multicore oracle in the literature are structurally blind to them. Four of the hardest bugs in this project's history were hangs.

(3) **Structural.** SWMR on the live L1 tag arrays, every cycle [15]. Fires at the transaction, not at the symptom ~10^8 cycles later.

(4) **Architectural.** Committed state versus a golden model, excluding legitimately racy commits. The strongest oracle when a reference exists.

Liveness and perturbation interact, and getting this wrong invalidates every result: a hart Kairos is deliberately holding is *supposed* to stop retiring. Both liveness layers therefore count only *unstalled* cycles. A livelock verdict additionally requires that no hart has been held for more than 1/8 of the elapsed run; past that the run is a `timeout`, which is not a finding. Without that discount the first four-policy sweep reported seven livelock "bugs", all of them Kairos holding harts. The soundness argument constrains which *states* a stall can produce; it says nothing about what an *oracle* may infer from a run the tool itself stalled into silence.

Known limitation: PCT stalls all but one hart, so on four harts every hart is held far more than 1/8 of any run and the livelock layer cannot fire under it. PCT contributes structural and wrong-result findings, not liveness ones. That is the price of a rule chosen never to cry wolf.

4.4 Coverage

Two metrics, sensitive to different things.

Distinct schedules counts 64-bit digests over the ordered sequence of cross-hart memory events. The digest excludes retirements and cycle numbers: two runs that produce the same event order at different absolute times are the same interleaving, and counting them separately is exactly the mistake a naive "hash the whole trace" metric makes.

Order pairs are the observed (*hart A touched line L before hart B*) orderings. Coarser, harder to saturate by accident, and the number to look at when someone asks whether "distinct schedules" is doing real work. Section 5 shows PCT leading on the first metric and losing on the second.

4.5 Reduction and replay

A campaign that finds a bug hands over a schedule holding harts thousands of times across millions of cycles. Kairos reduces it. The failing run is re-recorded as stall spans [*start*, *end*) *mask*. Delta debugging [18] drops spans until no remaining span can be dropped without losing the failure; a second pass binary-searches each surviving span from both ends. The predicate is "does this still reproduce the *same* verdict", not merely "does this still fail"—reducing a hang into a wrong-result failure would silently retarget the reduction onto a different bug.

Delta debugging normally struggles with concurrency because the predicate is flaky. Here it is not: the DUT is a cycle-accurate simulator and the candidate schedule is fully specified, so the predicate is a pure function. The same determinism that makes RTL simulation blind to interleavings is what makes reduction over them exact. The output is a five-line `.ksched` file that replays on any host, after the RTL changes, without a seed.

5 Evaluation

Setup. Chiron is a 4-core RV64IMA out-of-order processor with AXI-ACE snoops, private L1s and a shared L2, simulated under Verilator. Kairos links the fast (no-trace, threaded) model. Workloads are the in-tree SMP micros. Campaigns calibrate against an unperturbed run and refuse to start if that run does not pass. Self-tests of the design-independent half assert 100,052 checks against a mock DUT in one second, and are mutation-tested against nine deliberate regressions.

Compiled out, the generated Verilog differs from the pre-Kairos design in 0 lines once Chisel provenance comments are stripped (21,216 raw comment-only differences, from line-number shift).

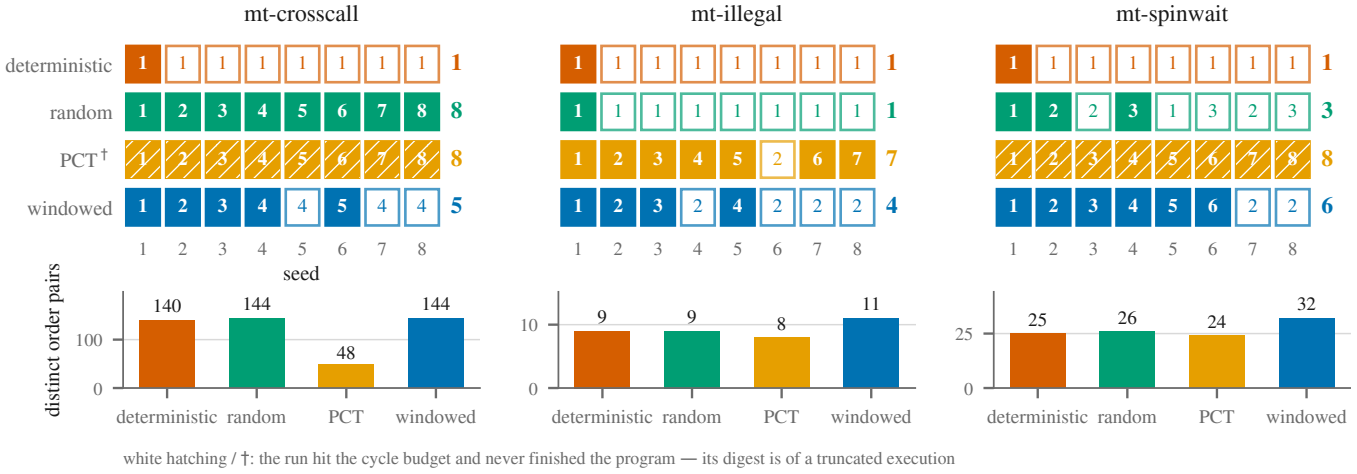

Figure 4: Schedule coverage, audited run by run. Top: each cell is one run; filled is an interleaving not seen before in that row, hollow is a repeat of class *n*, and white hatching marks a run that hit the cycle budget without finishing the program. The deterministic row *is* the paper's thesis. PCT leads or ties on distinct schedules—but on `mt-spinwait` and `mt-crosscall` every one of its runs is hatched, so those counts are over truncated executions. Bottom: order-pair coverage, the coarser metric. PCT produces *fewer* distinct cross-hart orderings on `mt-crosscall` (48 vs. 144): serialising harts explores different schedules without exploring different orderings.

Table 2: RTL and simulation cost. The 13% is the SWMR tag scan used as an oracle, not the stall bit.

Build	vs. original netlist	Gates
compiled out	0 lines	ISA 84/84, SMP 8/8, benches 5/5
in, mask = 0	1 bit + AND / core	same tests, bit-identical
+ tag scan	same	36.5 vs. 42 K cyc/s (−13%)

Compiled in with the mask held at zero, every SMP micro passes exactly as under the ordinary regression (Table 2).

5.1 Does a deterministic simulator explore one interleaving?

Yes. Figure 4 audits the claim run by run. Four policies, three workloads, eight seeds (96 runs). The `deterministic` row is one filled cell and seven hollow ones, on every workload. Distinct-schedule counts: `mt-illegal` (1.4 K cyc) 1 / 1 / 7 / 4; `mt-spinwait` (6.6 K) 1 / 3 / 8 / 6; `mt-crosscall` (390 K) 1 / 8 / 8 / 5, for deterministic / random / PCT / windowed.

On the current tree, after the races below were fixed, twelve deterministic runs of `mt-spinwait` still produce one digest and 29 order pairs; twelve windowed runs produce seven digests and 35 order pairs, and all twelve complete. The wall is a property of the simulator, not of a particular binary.

5.2 Which policy finds what?

Table 3 is the same-DUT comparison: four policies, three workloads, eight seeds (96 runs), one budget. We cannot port DifuzzRTL or Cascade onto this 4-core ACE out-of-order design—Cascade notes that instruction-granular lockstep on an OoO CPU is a non-trivial port, and said the same of TheHuzz when it was not open-source [14]—so we do not invent a number for them. The empty cell in Figure 2 is the positioning: no published tool varies the schedule of a fixed program on multicore RTL and observes cache state. The experiment we can run is this ablation.

Table 3: Same-DUT ablation: 4 policies × 3 workloads × 8 seeds, one budget. Per-cell order is `illegal/spinwait/crosscall`. A finding is a stall-aware livelock; a timeout is not a finding.

Policy	Distinct schedules	Runs completed	Findings
deterministic	1 / 1 / 1	8 / 8 / 8	0
random delay	1 / 3 / 8	8 / 8 / 8	0
PCT	7 / 8 / 8	8 / 0 / 0†	0‡
windowed	4 / 6 / 5	8 / 7 / 8	1

†PCT holds three of four harts, so the workload runs ∼100× slower: 16 of its 24 runs hit the budget, and those digests are of *truncated* executions (hatched in Figure 4). ‡The liveness layer cannot fire under PCT at all (Section 4.3). Finding 2 is absent here: it came from `windowed:400` on `mt-illegal` in 4 runs (5.6×10^3 cyc), outside this ablation—`windowed:2000` passes that workload 8/8.

Random delay is a weak strawman on short workloads: 1/8 distinct on `mt-illegal`, competitive only on the 390 K-cycle workload, where a 2%-per-cycle coin flip has 390,000 cycles to land somewhere useful. Per-cycle randomisation targets nothing.

PCT explores most and finds least, and the reason belongs in the text rather than a footnote. It leads or ties on distinct schedules everywhere—but on two of the three workloads it never finished the program: holding three of four harts costs ∼100× in run length, so 16 of its 24 runs ended at the budget and those digests are of partial executions. Its order-pair count is also *lower* on `mt-crosscall`, 48 against 144, because serialising harts visits many schedules without visiting many orderings. A single distinct-schedule number would have shown PCT winning three times over. That is an argument about the coverage model, not against PCT.

Windowed delay found both bugs: one contiguous delay of one named hart, the corner of the schedule space both findings lived in.

The experiment that did not work. We do not claim a 10^3–$10^4\times$ improvement on the dual-Unique bug, and we ran the experiment

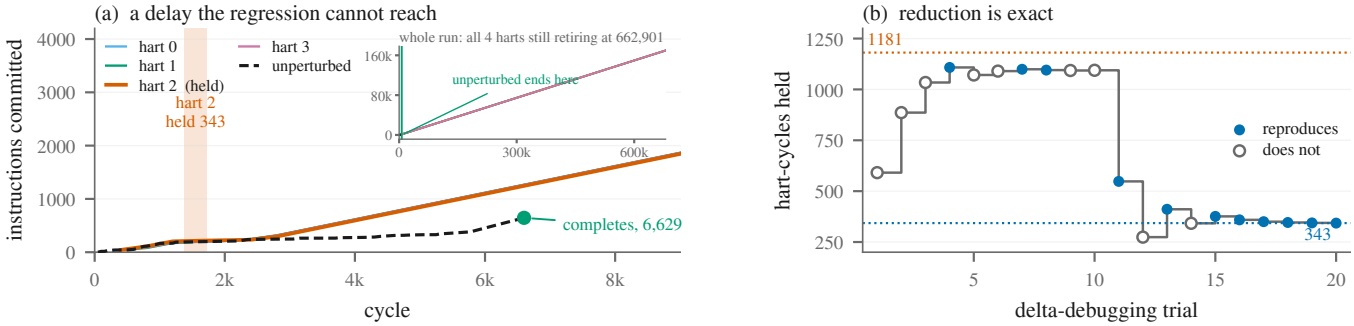

Figure 5: Finding 1, end to end. (a) A 343-cycle delay of hart 2, early in a 6,629-cycle workload. The unperturbed run finishes; the perturbed run is still retiring at 100× that length. (b) Delta debugging over that one span. The predicate is a pure function of the schedule, so every trial is tested once and the answer never changes.

that would have supported it. On the buggy RTL of Section 3, rebuilt with the hook, all four policies over `mt-fencei` and `mt-crosscall`, 12 seeds each, gave **96 runs and 0 findings**—while PCT reached 12/12 distinct schedules and windowed 8/12. Diversity went up; the bug still did not fire within $\sim 10^6$ cycles, against the 8.7×10^8 the Linux boot needed. That measured negative is the evidence for Section 6: a whole-core stall delays everything a hart does, and this race needs one walker writeback to outlive one peer's refill of one line.

5.3 Findings

Kairos produced two findings (Table 4). Both are software races in this repository's own test infrastructure. Both are now fixed and confirmed by A/B replay. Neither is an RTL bug. A fuzzing paper that misattributes its findings loses on the first reviewer who checks one.

Table 4: Findings. Software races, not RTL bugs. Both new relative to this repository; both fixed and confirmed by A/B replay.

#	Workload	Location	Class	Detected	Reduced
1	`mt-spinwait`	`.sbss` / `sd`	SW race	8 seeds	343 cyc, h2
2	`mt-illegal`	`crt.S` flag	SW race	4×1404 cyc	220 cyc, h2

Finding 2 — a latent race in the project's own boot code. Found on `mt-illegal` (baseline 1,404 cycles) under `windowed:400`, shrunk to hart 2 held for 220 cycles at cycle 284. All four harts spin forever in `crt.S`: every hart stores 1 to `hart_init_sync`, hart 0 later stores 0 to release everyone, and nothing orders a slow hart's store against that release. If any hart is delayed past hart 0's clear, its store re-arms the flag and every hart waits forever. The register dump is conclusive: `a2 = 1` on all four harts, including hart 0, the hart that executed the clearing store.

This defect is in code every SMP test in the repository links. It survived years of a green deterministic regression because a deterministic simulator always produces the one interleaving in which hart 0's `.bss` loop happens to take long enough. It was found in four runs of a 1,404-cycle workload.

Finding 1 — three compounding bugs around a barrier. Figure 5 is the anatomy. Under `windowed:2000`, seed 3, delaying hart 2 leaves a 6,629-cycle workload still running at 662,901 cycles with all four harts retiring inside their first barrier. The shrinker reduced 1,181 hart-cycles of delay to 343, in 20 simulation runs (Figure 5b). The cause was three defects stacked: the linker script did not cover `.sbss`, so the shared barrier state sat past `_ebss` and was never zeroed; every hart "initialised" it with an unsynchronised store at thread entry; and that store was an 8-byte `sd` on an `int *`, clobbering the adjacent sense flag. After the fix, replaying the same 343-cycle schedule on the new binary completes in 8,213 cycles with all four harts at `exit`.

Attribution discipline. Kairos reports "this schedule wedges the machine". Triage decides whether the fault is in the hardware or the software. The project's own history says why that matters: `mt-llist` spent weeks as a suspected coherence bug before turning out to be its own CAS loop fencing itself into a livelock. We do not count either finding as an RTL bug, and we do not inflate the finding count with timeouts.

6 Limitations

Whole-core stalls are too coarse for at least one line-level race, and we measured that rather than suspecting it: 96 exploring runs on the RTL with the dual-Unique fix reverted produced no finding (Section 5.2). That bug needs a walker writeback to outlive a peer's refill of a particular line; stalling a hart delays everything that hart does. Optional next knobs—snoop-response delay, arbiter priority—act at the transaction rather than the core, are equally sound by the same argument, and are not evaluated here. PCT's bug-depth guarantee does not transfer cleanly (Section 4.2), and PCT cannot contribute liveness findings under the conservative oracle rule. The evaluation is one DUT. Timeouts are not findings; a campaign that under-provisions its budget reports nothing, which is the honest outcome.

7 Conclusion

RTL simulation of a multicore explores one interleaving per program. We measured that wall, on a real coherence bug that five directed tests could not distinguish from its fix, and we lifted it with one bit per core and a host-side scheduler. The failures are real because a stall is always legal; the disabled design is the original design because the netlist matches to the line; the findings replay because the DUT is deterministic and the schedule is a file. Schedule exploration is not a better fuzzer. It is the other axis.

References

[1] Jade Alglave, Luc Maranget, and Michael Tautschnig. 2014. Herding Cats: Modelling, Simulation, Testing, and Data Mining for Weak Memory. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 36, 2 (2014).

[2] Sebastian Burckhardt, Pravesh Kothari, Madanlal Musuvathi, and Santosh Nagarakatte. 2010. A Randomized Scheduler with Probabilistic Guarantees of Finding Bugs. In *Proceedings of the 15th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.

[3] Sadullah Canakci, Leila Delshadtehrani, Furkan Eris, Michael Taylor Le, Ajay Joshi, and Manuel Egele. 2023. ProcessorFuzz: Processor Fuzzing with Control and Status Registers Guidance. In *IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*.

[4] Marco Elver and Vijay Nagarajan. 2016. McVerSi: A Test Generation Framework for Fast Memory Consistency Verification in Simulation. In *IEEE International Symposium on High Performance Computer Architecture (HPCA)*.

[5] Sudheendra Hangal, Durgam Vahia, Chaiyasit Manovit, Juin-Yeu Joseph Lu, and Sridhar Narayanan. 2004. TSOtool: A Program for Verifying Memory Systems Using the Memory Consistency Model. In *Proceedings of the 31st Annual International Symposium on Computer Architecture (ISCA)*.

[6] Jaewon Hur, Suhwan Song, Dongup Kwon, Eunjin Baek, Jangwoo Kim, and Byoungyoung Lee. 2021. DifuzzRTL: Differential Fuzz Testing to Find CPU Bugs. In *IEEE Symposium on Security and Privacy (S&P)*.

[7] IEEE. 2020. *IEEE Standard for Universal Verification Methodology Language Reference Manual.* IEEE Std 1800.2-2020.

[8] Rahul Kande, Addison Crump, Garrett Persyn, Patrick Jauernig, Ahmad-Reza Sadeghi, Aakash Tyagi, and Jeyavijayan Rajendran. 2022. TheHuzz: Instruction Fuzzing of Processors Using Golden-Reference Models for Finding Software-Exploitable Vulnerabilities. In *31st USENIX Security Symposium*.

[9] Kevin Laeufer, Jack Koenig, Donggyu Kim, Jonathan Bachrach, and Koushik Sen. 2018. RFUZZ: Coverage-Directed Fuzz Testing of RTL on FPGAs. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*.

[10] Doowon Lee and Valeria Bertacco. 2017. MTraceCheck: Validating Non-Deterministic Behavior of Memory Consistency Models in Post-Silicon Validation. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*.

[11] Yifei Ma et al. 2025. DiffTest-H: Toward Cost-Effective CPU Verification with Expanding Architectural States. In *58th IEEE/ACM International Symposium on Microarchitecture (MICRO)*.

[12] Madanlal Musuvathi and Shaz Qadeer. 2007. Iterative Context Bounding for Systematic Testing of Multithreaded Programs. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*.

[13] Madanlal Musuvathi, Shaz Qadeer, Thomas Ball, Gerard Basler, Piramanayagam Arumuga Nainar, and Iulian Neamtiu. 2008. Finding and Reproducing Heisenbugs in Concurrent Programs. In *8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

[14] Flavien Solt, Katharina Ceesay-Seitz, and Kaveh Razavi. 2024. Cascade: CPU Fuzzing via Intricate Program Generation. In *33rd USENIX Security Symposium*.

[15] Daniel J. Sorin, Mark D. Hill, and David A. Wood. 2011. *A Primer on Memory Consistency and Cache Coherence.* Morgan & Claypool.

[16] Jinyan Xu, Yiyuan Liu, Sirui He, Haoran Lin, Yajin Zhou, and Cong Wang. 2023. MorFuzz: Fuzzing Processor via Runtime Instruction Morphing enhanced Synchronizable Co-simulation. In *32nd USENIX Security Symposium*.

[17] Yinan Xu, Zihao Yu, Dan Tang, Guokai Chen, Lu Chen, Lingrui Gou, Yue Jin, Qianruo Li, Xin Li, Zuojun Li, et al. 2022. Towards Developing High Performance RISC-V Processors Using Agile Methodology. In *55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*.

[18] Andreas Zeller and Ralf Hildebrandt. 2002. Simplifying and Isolating Failure-Inducing Input. *IEEE Transactions on Software Engineering* 28, 2 (2002).